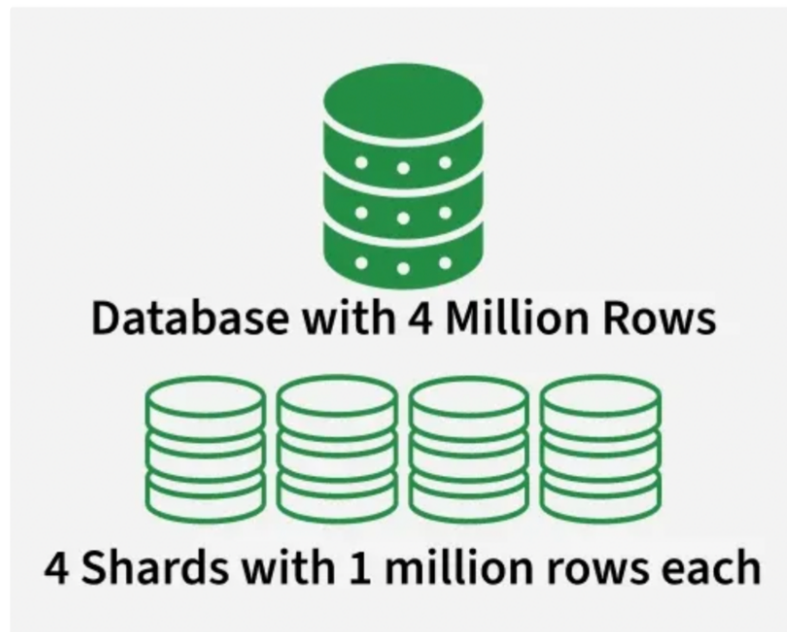


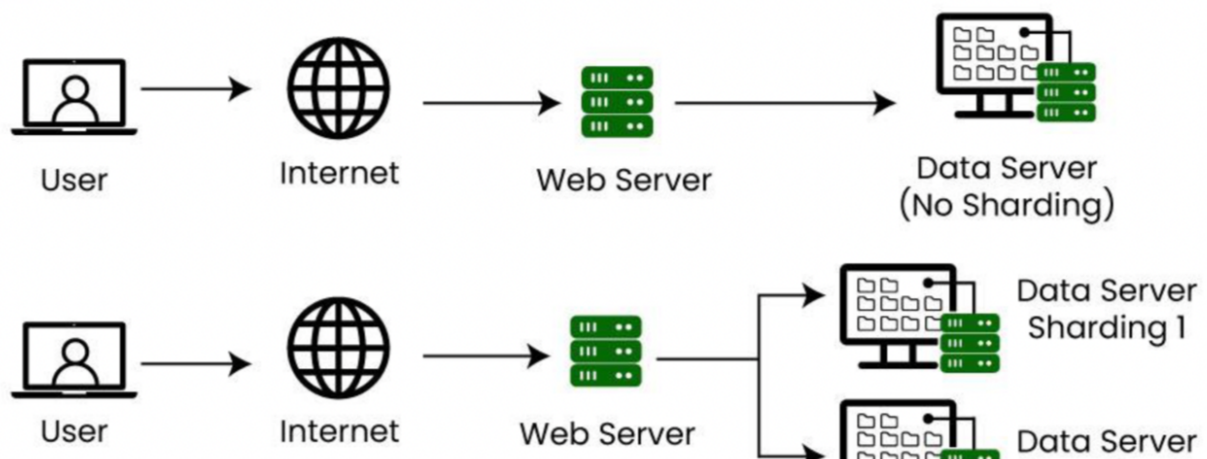
Database sharding is a technique for horizontal scaling of databases, where the data is split across multiple database instances, or shards to improve performance and reduce the impact of large amounts of data on a single database.



It is basically a **database architecture pattern** in which we split a large dataset into smaller chunks (logical shards) and we store/distribute these chunks in different machines/database nodes (physical shards).

- Each chunk/partition is known as a "**shard**" and each shard has the same database schema as the original database.
- We distribute the data in such a way that each row appears in exactly one shard.
- It's a good mechanism to improve the **scalability** of an application.

Sharding



Methods of Sharding

1. Key Based Sharding

Key Based Sharding is a technique is also known as **hash-based** sharding. Here, we take the value of an entity such as customer ID, customer email, IP address of a client, zip code, etc and we use this value as an input of the **hash function**. This process generates a **hash value** which is used to determine which shard we need to use to store the data.

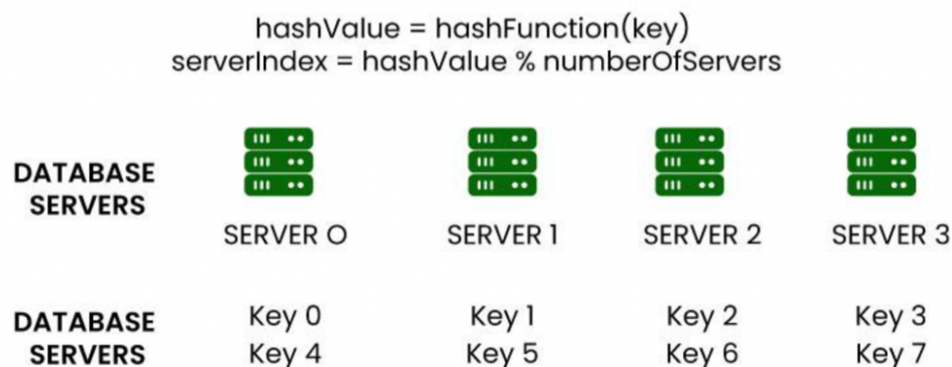
- We need to keep in mind that the values entered into the hash function should all come from the **same column** (shard key) just to ensure that data is placed in the correct order and in a consistent manner.
- Basically, shard keys act like a primary key or a unique identifier for individual rows.

For example:

You have 3 database servers and each request has an application id which is incremented by 1 every time a new application is registered.

To determine which server data should be placed on, we perform a modulo operation on these applications id with the number 3. Then the remainder is used to identify the server to store our data.

Key-Based Sharding



Sharding/ Distributing Data Across Several Database Servers

Advantages of Key Based Sharding:

- Key-based sharding assigns each key to a specific shard, ensuring uniform and consistent data distribution.
- It can be optimized to efficiently handle queries over consecutive key ranges.

Disadvantages of Key Based Sharding:

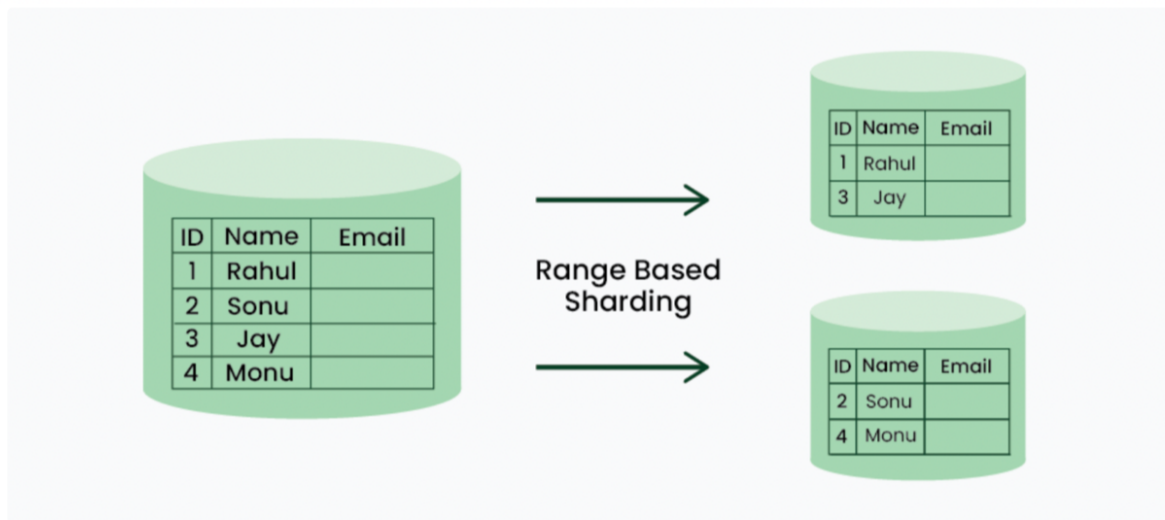
- Uneven data distribution can occur if the sharding key isn't well-distributed.
- Scalability may be limited when certain keys receive heavy traffic or data is skewed.

- Choosing the right sharding key is crucial for effective sharding.

2. Horizontal or Range Based Sharding

In **Horizontal or Range Based Sharding**, we divide the data by separating it into different parts based on the range of a specific value within each record. Let's say you have a database of your online customers' names and email information. You can split this information into two shards.

- In one shard you can keep the info of customers whose first name starts with A-P
- In another shard, keep the information of the rest of the customers.



Advantages of Range Based Sharding:

- **Scalability:** Horizontal or range-based sharding allows for seamless scalability by distributing data across multiple shards, accommodating growing datasets.
- **Improved Performance:** Data distribution among shards enhances query performance through parallelization, ensuring faster operations with smaller subsets of data handled by each shard.

Disadvantages of Range Based Sharding:

- **Complex Querying Across Shards:** Coordinating queries involving multiple shards can be challenging.
- **Uneven Data Distribution:** Poorly managed data distribution may lead to uneven workloads among shards.

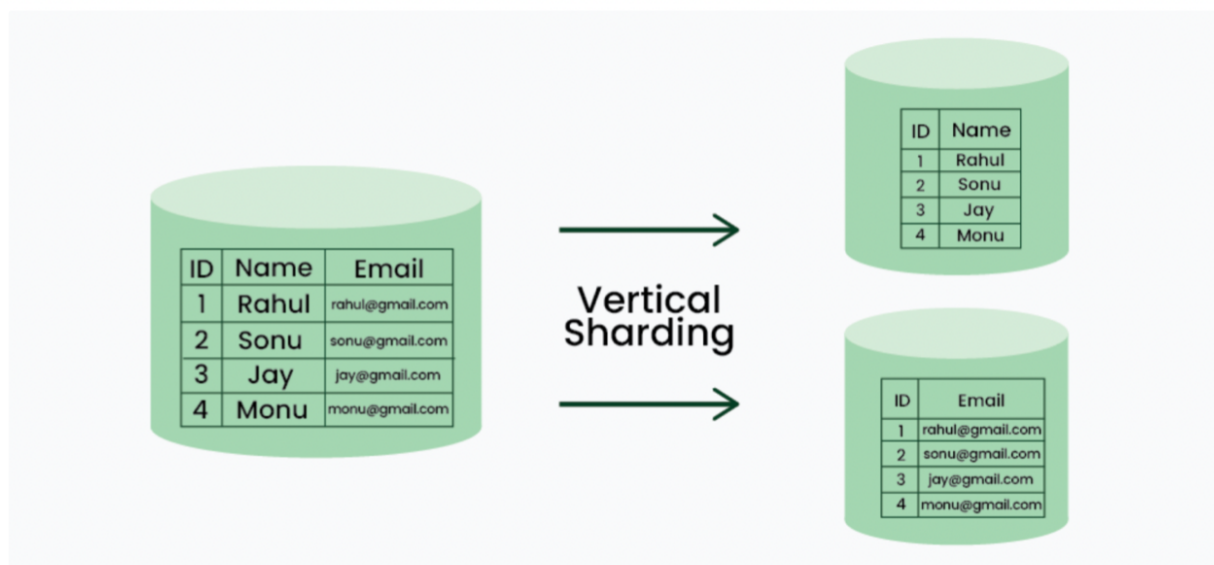
3. Vertical Sharding

In **Vertical Sharding**, we split the entire column from the table and we put those columns into new distinct tables. Data is totally independent of one partition to the other ones. Also, each partition holds both distinct rows and columns. We can split different features of an entity in different shards on different machines.

For example:

On Twitter users might have a profile, number of followers, and some tweets posted by

On Twitter users might have a profile, number of followers, and some tweets posted by his/her own. We can place the user profiles on one shard, followers in the second shard, and tweets on a third shard.



Advantages of Vertical Sharding:

- **Query Performance:** Vertical sharding can improve query performance by allowing each shard to focus on a specific subset of columns. This specialization enhances the efficiency of queries that involve only a subset of the available columns.
- **Simplified Queries:** Queries that require a specific set of columns can be simplified, as they only need to interact with the shard containing the relevant columns.

Disadvantages of Vertical Sharding:

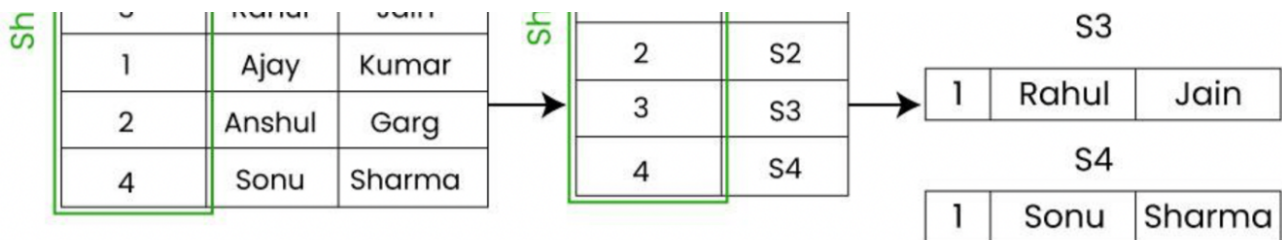
- **Potential for Hotspots:** Certain shards may become hotspots if they contain highly accessed columns, leading to uneven distribution of workloads.
- **Challenges in Schema Changes:** Making changes to the schema, such as adding or removing columns, may be more challenging in a vertically sharded system. Changes can impact multiple shards and require careful coordination.

4. Directory-Based Sharding

In **Directory-Based Sharding**, we create and maintain a **lookup service** or lookup table for the original database. Basically we use a **shard key** for lookup table and we do **mapping** for each entity that exists in the database. This way we keep track of which database shards hold which data.

Directory-Based Sharding





The lookup table holds a static set of information about where specific data can be found. In the above image, you can see that we have used the delivery zone as a shard key:

- Firstly the client application queries the lookup service to find out the shard (database partition) on which the data is placed.
- When the lookup service returns the shard it queries/updates that shard.

Advantages of Directory-Based Sharding:

- **Flexible Data Distribution:** Directory-based sharding allows for flexible data distribution, where the central directory can dynamically manage and update the mapping of data to shard locations.
- **Efficient Query Routing:** Queries can be efficiently routed to the appropriate shard using the information stored in the directory. This results in improved query performance.
- **Dynamic Scalability:** The system can dynamically scale by adding or removing shards without requiring changes to the application logic.

Disadvantages of Directory-Based Sharding:

- **Centralized Point of Failure:** The central directory represents a single point of failure. If the directory becomes unavailable or experiences issues, it can disrupt the entire system, impacting data access and query routing.
- **Increased Latency:** Query routing through a central directory introduces an additional layer, potentially leading to increased latency compared to other sharding strategies.

Ways to optimize database sharding for even data distribution

Here are some **simple ways to optimize database sharding for even data distribution**:

- **Use Consistent Hashing:** This helps distribute data more evenly across all shards by using a hashing function that assigns records to different shards based on their key values.
- **Choose a Good Sharding Key:** Picking a well-balanced sharding key is crucial. A key that doesn't create hotspots ensures that data spreads out evenly across all servers.
- **Range-Based Sharding with Caution:** If using range-based sharding, make sure the ranges are properly defined so that one shard doesn't get overloaded with more data than others.
- **Regularly Monitor and Rebalance:** Keep an eye on data distribution and rebalance shards when necessary to avoid uneven loads as data grows.
- **Automate Sharding Logic:** Implement automation tools or built-in database features that automatically distribute data and handle sharding to maintain balance across shards.

Alternatives to database sharding

Below are some of the alternatives to database sharding:

Below are some of the alternatives to database sharding.

1. **Vertical Scaling:** Instead of splitting the database, you can upgrade your existing server by adding more CPU, memory, or storage to handle more load. However, this has limits as you can only scale a server so much.
2. **Replication:** You can create copies of your database on multiple servers. This helps with load balancing and ensures availability, but can lead to synchronization issues between replicas.
3. **Partitioning:** Instead of sharding across multiple servers, partitioning splits data within the same server. It divides data into smaller sections, improving query performance for large datasets.
4. **Caching:** By storing frequently accessed data in a cache (like Redis or Memcached), you reduce the load on your main database, improving performance without needing to shard.
5. **CDNs:** For read-heavy workloads, using a Content Delivery Network (CDN) can offload some of the data access from your primary database, reducing the need for sharding.

Advantages of Sharding

Sharding offers many advantages in system design such as:

1. **Enhances Performance:** By distributing the load among several servers, each server can handle less work, which leads to quicker response times and better performance all around.
2. **Scalability:** Sharding makes it easier to scale as your data grows. You can add more servers to manage the increased data load without affecting the system's performance.
3. **Improved Resource Utilization:** When data is dispersed, fewer servers are used, reducing the possibility of overloading one server.
4. **Fault Isolation:** If one shard (or server) fails, it doesn't take down the entire system, which helps in better fault isolation.
5. **Cost Efficiency:** You can use smaller, cheaper servers instead of investing in a large, expensive one. As the system grows, sharding helps keep costs in control.

Disadvantages of Sharding

Sharding comes with some disadvantages in system design such as:

1. **Increased Complexity:** Managing and maintaining multiple shards is more complex than working with a single database. It requires careful planning and management.
2. **Rebalancing Challenges:** If data distribution becomes uneven, rebalancing shards (moving data between servers) can be difficult and time-consuming.
3. **Cross-Shard Queries:** Queries that need data from multiple shards can be slower and more complicated to handle, affecting performance.
4. **Operational Overhead:** With sharding, you'll need more monitoring, backups, and maintenance, which increases operational overhead.
5. **Potential Data Loss:** If a shard fails and isn't properly backed up, there's a higher risk of losing the data stored on that shard.